A compiler made of functions

CS413 Midterm 1 | Extended LAMBDA into pure lambda calculus

Numbers. Booleans. Pairs. Recursion.

The output keeps only three building blocks:

Var

a name

Lam

a function

App

a call

Build the program, run it, then decode the answer.

Source and target languages

SOURCE: MANY FORMS

Natural numbers and booleans
Functions, calls, and let
Arithmetic and comparisons
Conditionals and boolean ops
Pairs and recursive functions

TARGET: EXACTLY THREE FORMS

```
Var(name)
Lam(name, body)
App(fn, arg)

No numeric or boolean nodes.
```

Must represent source language using only three forms.

Architecture: one shared evaluator

Compilation constructs trees. Evaluation runs them.

SOURCE AST

lambda1_vp.py

>

COMPILER

compiler.py

>

TARGET AST

Var / Lam / App

>

DECODE

decoders.py

ENCODING LIBRARY

Closed pure terms used by the compiler

SHARED EVALUATOR: target_ast.py

Runs source ASTs, target terms, and observation contexts

Source result and decoded target result must agree.

Compilation follows the source tree

C[e] means the translation of e. Encoding names are expanded.

KEY RULES

```
Variable ……… C[x] = x
Lambda …… C[lam x. e] = lam x. C[e]
Application … C[e1 e2] = C[e1] C[e2]
Let …… C[let x = e1 in e2] = (lam x. C[e2]) C[e1]
Addition ……… C[a + b] = ADD C[a] C[b]
```

CompileVisitor handles every source constructor, without evaluation.

Church values are functions

These definitions become ordinary Var, Lam, and App nodes.

NATURAL NUMBERS

```
0 = lam s. lam z. z
3 = lam s. lam z. s (s (s z))
```

Apply s to z, n times.

BOOLEANS

```
TRUE  = lam t. lam f. t
FALSE = lam t. lam f. f
```

A boolean chooses a value.

Zero and false have the same structure: decode with num or bool.

Predecessor remembers two counts

Each step changes (previous, current) to (current, current + 1).

PAIR		PAIR		PAIR		PAIR
`(0, 0)`	>	`(0, 1)`	>	`(1, 2)`	>	`(2, 3)`

Church 3 repeats the step three times. FST returns 2.

```
SUB m n = n PRED m
```

2 - 5: 2 -> 1 -> 0 -> 0 -> 0 -> 0. Subtraction stops at zero.

Weak, left-to-right call-by-value

Three rules describe one evaluation strategy.

WEAK

Stop at a lambda. Do not enter its body until application.

LEFT-TO-RIGHT

Evaluate the function before the argument.

CALL-BY-VALUE

Finish the argument before applying the function.

Function value -> argument value -> substitution

(lam x. 0) LOOP still runs LOOP, even though x is unused.

Names must keep their meaning

Free names stay free. Inner bindings take priority.

CAPTURE AVOIDANCE

```
Replace x with y in:
lam y. x

Safe result:
lam y_1. y
```

NAMING AND SCOPE

Smallest free positive suffix:
y_1, y_2, ...
Let binds only its body.
Fix binds f and x.
Same names: parameter wins.

FV(C[e]) = FV(e). Common names like s, z, t, f, and x are allowed.

Delayed branches preserve control flow

A thunk is a function wrapper that postpones an expression.

CONDITIONAL TRANSLATION

```
(C[c] (lam u. C[t]) (lam v. C[f])) I
I = lam w. w
```

The boolean selects a wrapper; the final I runs it.

AND and OR delay their right operand in the same way.

if true then 1 else LOOP returns 1. The loop branch is never forced.

An encoded answer still needs decoding

observe() performs execution, then bounded observation.

EXECUTION BUDGET

Run the compiled program.
Stop at a lambda value.
Count beta-value steps.
Default: 50,000.

OBSERVATION BUDGET

Apply a pure context.
Inspect the final data shape.
Keep a separate step count.
Default: 50,000.

Pairs share one observation budget across all projections and components.

Results from the current implementation

17/17 examples pass: 14 required demonstrations plus 3 extras.

Program	Answer	Exec steps	Obs steps
2 + 3 * 4	14	4	26
2 - 5	0	87	2
factorial(4)	24	353	333
Fibonacci(5)	5	1794	45
nested pair	((2, True), 3)	4	27

Counts are measured beta-value steps, not timing guarantees.

Live demo: change the budget flags

Run from assigns/06/. Keep this slide beside the terminal.

```
python -B -m MySolution.demo
```

```
python -B -m MySolution.demo --execution-budget 100
```

```
python -B -m MySolution.demo --observation-budget 100
```

```
python -B -m MySolution.demo --execution-budget <exact>
```

Point out execution status, observation status, and the separate counts.

Tradeoffs and limitations

The design favors inspectable semantics over execution speed.

DESIGN TRADEOFFS

Predecessor with pair implementation is costly.

Budgets can stop valid programs early.

Decoder requires a “num” or “bool” input.

LIMITATIONS

Natural numbers only.

Decoder needs num/bool/pair kinds.

Large terms use time and memory.

Step limits do not cap elapsed time or memory; malformed detection is incomplete.